

ARQUITETURA ORIENTADA A OBJETOS II

POLIMORFISMO

Capacidade de um método ou entidade assumir muitas formas. Diferentes classes são tratadas como instâncias de uma única classe base, por meio de uma base comum.

Existem três tipos:

DE INCLUSÃO (SUBTIPAGEM)

É o tipo mais comum. A subclasse sobrescreve um método da sua classe base – um objeto da subclasse é tratado como objeto da classe base.

No exemplo a seguir, a classe “cão” é estendida da classe “animal”. A classe “animal” faz algum som, ao passo que a classe “cão”, que é a classe filha, mesmo tendo o mesmo método, em vez de imprimir “faz algum som”, imprime “latido”.

```
Classe Animal
  Método emitirSom()
    Imprimir "Faz algum som"

Classe Cão Extende Animal
  Método emitirSom()
    Imprimir "Latido"
```

PARAMÉTRICO

Definem funções ou tipos de dados de forma genérica, operando com qualquer tipo de dado.

```
Função genérica imprimir<T>(valor: T)
  Imprimir valor
```

SOBRECARGA (AD-HOC)

Várias funções com o mesmo nome mas diferentes assinaturas (tipo ou número de parâmetros). No exemplo a seguir, há duas funções “somar”, mas, na primeira, o tipo do parâmetro é inteiro, e na segunda é real, ou seja, há uma diferença na assinatura.

```
Função somar(a: Inteiro, b: Inteiro)
  Retornar a + b

Função somar(a: Real, b: Real)
  Retornar a + b
```



DIRETO DO CONCURSO

03. (CESPE-CEBRASPE/2022/BANRISUL/DESENVOLVIMENTO DE SISTEMAS) Acerca dos padrões de projeto em arquitetura de software, julgue o próximo item.

Em um projeto orientado a objetos, a decomposição do sistema em objetos é influenciada por fatores como encapsulamento, granularidade e desempenho.



Está correto.



5m

PRINCÍPIOS DE PROJETO

Princípios de projeto ou princípios de design de software são diretrizes que ajudam desenvolvedores a criar sistemas de software que são fáceis de manter, entender e expandir.

São aplicáveis em qualquer tipo de arquitetura de software ou programação:

- DRY (Don't Repeat Yourself);
- YAGNI (You Ain't Gonna Need It); e
- KISS (Keep It Simple, Stupid).

O DRY tem como princípio não escrever mais de uma vez no código algo que poderia ser replicado.

Por exemplo, em vez de escrever os mesmos métodos da classe animal para a classe cachorro, esses métodos seriam simplesmente herdados. Dessa forma, eles também seriam corrigidos simultaneamente.

O YAGNI serve para não adicionar elementos no projeto que não são necessários no momento. O KISS prevê que o sistema seja mantido da forma mais simples possível.

É específico de Arquitetura Orientada a Objetos: SOLID, um acrônimo para cinco princípios de design orientado a objeto.

SOLID

- **Single Responsibility Principle** (Princípio da Responsabilidade Única): uma classe deve ter apenas uma responsabilidade. Do contrário, a modularidade seria comprometida.
- **Open/Closed Principle** (Princípio Aberto/Fechado): entidades de software devem estar abertas para extensão, mas fechadas para modificação.
- **Liskov Substitution Principle** (Princípio da Substituição de Liskov): objetos de um programa devem ser substituíveis por instâncias de seus subtipos sem alterar a correção do programa.



10m

Nesse caso, as instâncias dos subtipos não devem alterar a correção de algum programa, no sentido de que eles funcionam de uma forma modular.

- **Interface Segregation Principle** (Princípio da Segregação de Interface): muitas interfaces específicas são melhores que uma interface única. Se todos os elementos fossem interligados, a alteração em uma parte do código implicaria outras partes do sistema.
- **Dependency Inversion Principle** (Princípio da Inversão de Dependência): dependa de abstrações, não de implementações concretas.



DIRETO DO CONCURSO

04. (CESPE-CEBRASPE/2022/BANRISUL/DESENVOLVIMENTO DE SISTEMAS) Acerca dos padrões de projeto em arquitetura de software, julgue o próximo item.

O princípio da segregação de interface dos padrões SOLID define que uma classe deve possuir somente uma operação para ser executada.



Não, muitas interfaces específicas são melhores que uma interface única.

PADRÕES DE PROJETO COMPORTAMENTAIS

São uma categoria de padrões de design em programação orientada a objetos que lidam com a comunicação eficaz e a distribuição de responsabilidades entre objetos.

- **Observer:** utilizado para criar um sistema onde objetos se inscrevem para observar e reagir a eventos em outros objetos.
 - **Exemplo:** aplicativo de notícias com opção de inscrição para receber notificações.
- **Strategy:** permite a configuração de diferentes algoritmos para serem usados em um objeto, tornando-os intercambiáveis.
 - **Exemplo:** aplicativo de navegação que pode calcular diferentes rotas.
- **Command:** transforma uma solicitação em um objeto autônomo, permitindo operações como enfileiramento e desfazer ações.
 - **Exemplo:** controle que manda comandos para aparelhos eletrônicos.
- **State:** permite que um objeto mude seu comportamento quando seu estado interno muda, parecendo alterar sua classe.
 - **Exemplo:** gerenciador de download (pode pausar ou cancelar quando está em andamento).



15m

- **Iterator:** permite percorrer elementos de uma coleção sem expor sua representação subjacente.
 - **Exemplo:** listas, pilhas, árvores, etc.
- **Memento:** permite que o estado de um objeto seja salvo e restaurado em um momento posterior.
 - **Exemplo:** comando “desfazer” em editores de texto.



DIRETO DO CONCURSO

05. (FGV/2024/CGM DE BELO HORIZONTE/MG/AUDITOR INTERNO/CIÊNCIA DA COMPUTAÇÃO/MANHÃ) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos.

Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.



Os padrões de projeto comportamentais são: iterator, memento e observer.

PADRÕES GRASP

- **General Responsibility Assignment Software Patterns:** são um conjunto de princípios na programação orientada a objetos. Eles solucionam problemas comuns de atribuição de responsabilidades em modelos de software.
- **Especialista em Informação:** atribui responsabilidades à classe que detém as informações necessárias para realizar uma tarefa específica.
- **Criador:** determina que uma classe que contém ou usa frequentemente outra deve ser responsável por criar instâncias dessa outra classe.
- **Controlador:** centraliza o processamento de solicitações, atribuindo a responsabilidade a uma classe que representa um caso de uso específico.
- **Baixo Acoplamento:** promove independência entre classes para aumentar a reutilização e a flexibilidade do código.
- **Alta Coesão:** encoraja o design de classes com responsabilidades bem definidas e focadas, melhorando a compreensibilidade e reduzindo erros.



20m



DIRETO DO CONCURSO

06. (CESPE-CEBRASPE/2022/BANRISUL/QUALITY ASSURANCE (QA) E ANALISTAS DE TESTE) Julgue o item a seguir, a respeito dos padrões GRASP (general responsibility assignment software patterns).

Entre os padrões definidos pelo GRASP, destacam-se baixa coesão e alto acoplamento.



Não, destacam-se baixo acoplamento e alta coesão.

07. (CESPE-CEBRASPE/2022/DPE-RO/ANALISTA DA DEFENSORIA PÚBLICA/PROGRAMAÇÃO) O GRASP (general responsibility assignment software patterns) define princípios básicos padrões de projetos orientados a objetos. Considere os seguintes questionamentos, feitos no âmbito de um sistema escolar onde se conhece a média total do resultado de um aluno.

- Para se conhecer a média total do resultado de um aluno, qual princípio vai se direcionar para encontrar a classe de objetos adequada para receber essa responsabilidade?
- Nesse caso, qual princípio GRASP procura identificar a classe de objetos que tem a informação necessária para a determinação da média?

Assinale a opção que apresenta o princípio GRASP presente nos referidos questionamentos.

- a) especialista na informação (information expert)
- b) acoplamento baixo (low coupling)
- c) coesão alta (high cohesion)
- d) controlador (controller)
- e) criador (creator)



O especialista na informação analisa as informações necessárias para se fazer o direcionamento dessa classe para o recebimento da responsabilidade, que, nesse caso, é de calcular a média dos alunos.

08. (FGV/2022/SEFAZ-AM/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO DA FAZENDA ESTADUAL/TARDE) Os padrões GRASP descrevem princípios fundamentais de atribuição de responsabilidades em projetos de software orientado a objeto.

De acordo com esses padrões, o polimorfismo é um princípio orientador para atribuir responsabilidades



25m

- a) a classes artificiais que não representem nada no domínio do problema de maneira a obter uma coesão alta.
- b) a classes abstratas que representem o comportamento de classes concretas para permitir ao sistema lidar com vários tipos de maneira homogênea.
- c) a uma classe controladora global para centralizar o tratamento de todas as mensagens e eventos do sistema.
- d) a classes que têm a informação necessária para satisfazer à responsabilidade e colaborar para que o acoplamento permaneça alto.
- e) a classes estáticas, altamente coesas e que são utilizadas no sistema sem a necessidade de instanciar objetos.



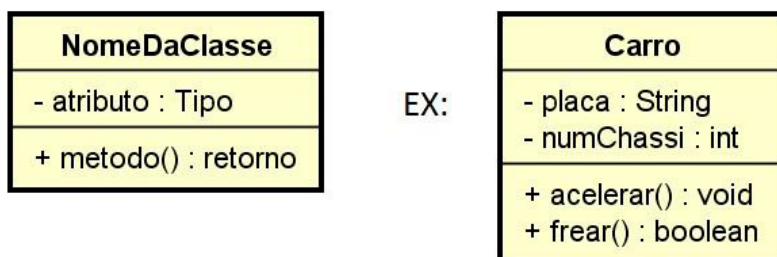
De acordo com esses padrões, o polimorfismo é um princípio orientador para atribuir responsabilidades a classes abstratas que representem o comportamento de classes concretas para permitir ao sistema lidar com vários tipos de maneira homogênea.

UML (UNIFIED MODELING LANGUAGE)

É uma linguagem que foi criada como um modelo padronizado para descrever uma abordagem de programação orientada ao objeto. Os diagramas de classes são componentes básicos da UML.

Facilita o entendimento da arquitetura de software, pois proporciona representação gráfica das estruturas e dos comportamentos do sistema.

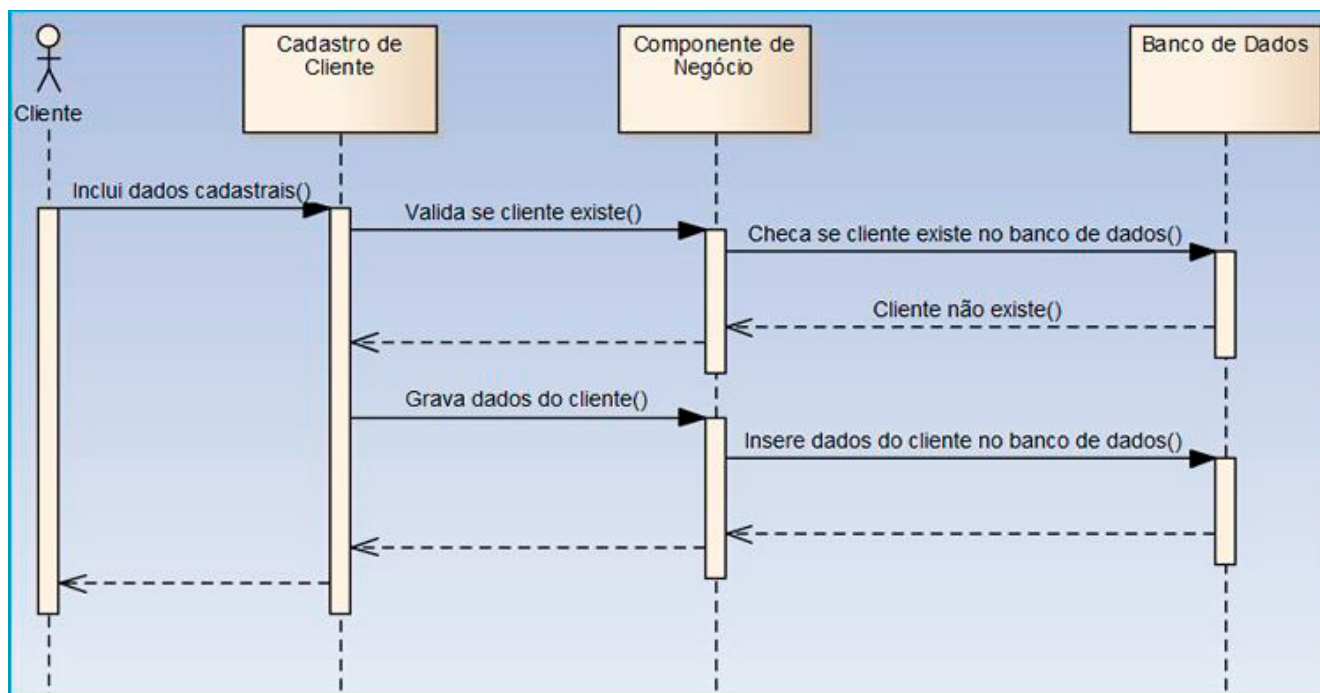
Diagramas de classes mostram a estrutura das classes e suas relações:



Diagramas de casos de uso descrevem as funcionalidades do sistema e suas interações com os usuários. O exemplo a seguir representa uma clínica, o relacionamento do paciente com o secretário e com o médico.



Diagramas de sequência e diagramas de atividades ilustram o fluxo de operações. No exemplo a seguir, o cliente realiza o seu cadastro incluindo seus dados cadastrais. Depois, há uma validação, para verificar se o cliente existe. Por fim, os dados dos clientes são inseridos no banco de dados.



DIRETO DO CONCURSO

09. (UFV/2024/UFV-MG/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO) Em projetos de sistemas, a UML (Unified Modeling Language) é amplamente utilizada para visualizar e documentar a estrutura e o comportamento do sistema. Considerando a UML, o diagrama UML que é comumente usado para representar a interação entre objetos durante a execução de um cenário específico é:

- a) Diagrama de Atividades.
- b) Diagrama de Sequência.
- c) Diagrama de Caso de Uso.
- d) Diagrama de Entidade-Relacionamento.



O diagrama UML que é comumente usado para representar a interação entre objetos durante a execução de um cenário específico é o Diagrama de Sequência.

GABARITO

03. C**04. E****05. c****06. E****07. a****08. b****09. b**

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pela professora Ana Júlia Batista de Souza.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
